

Programación III — Guía de Ejercicios Prácticos

Tema: Conceptos Básicos de Algoritmos, Eficiencia y Notación Asintótica

Asignatura: Programación III

Modalidad: Trabajo Práctico Individual / Grupal

Indicaciones Generales:

Esta guía contiene actividades prácticas diseñadas para afianzar los conceptos abordados en la clase teórica: definición de algoritmos, análisis de eficiencia en el peor caso, conteo de operaciones elementales y clasificación asintótica. Completa los ejercicios en el tiempo asignado durante el taller.

Sección 1: Conteo de Operaciones y Análisis de Código

EJERCICIO 1

Cálculo de Operaciones Elementales y Peor Caso

Dado el siguiente algoritmo escrito en Java para verificar si un arreglo contiene elementos duplicados:

```
public boolean contieneDuplicados(int[] v) {  
    int n = v.length;           // LÍNEA 2  
    for (int i = 0; i < n; i++) { // LÍNEA 3  
        for (int j = i + 1; j < n; j++) { // LÍNEA 4  
            if (v[i] == v[j]) { // LÍNEA 5  
                return true; // LÍNEA 6  
            }  
        }  
    }  
    return false; // LÍNEA 9  
}
```

Consignas:

1. Identifica cuál es la instancia de datos de entrada que representa el **peor caso** de ejecución para este algoritmo.
2. Calcula la cantidad exacta de operaciones elementales $T(n)$ ejecutadas en el peor caso en función del tamaño del arreglo n .
3. Determina la notación asintótica Big-O (O) que clasifica la complejidad temporal del algoritmo en el peor caso.

EJERCICIO 2

El Dilema del Condicional y la Regla del Máximo

Analiza el siguiente método que procesa una matriz cuadrada de tamaño $n \times n$ de dos formas diferentes según una bandera booleana:

```
public void procesarMatriz(int[][] matriz, boolean modoRapido) {  
    int n = matriz.length;  
  
    if (modoRapido) {  
        // RAMA A: Recorrido de la diagonal principal  
        for (int i = 0; i < n; i++) {  
            System.out.println(matriz[i][i]);  
        }  
    } else {  
        // RAMA B: Recorrido completo de la matriz  
        for (int i = 0; i < n; i++) {  
            for (int j = 0; j < n; j++) {  
                System.out.println(matriz[i][j]);  
            }  
        }  
    }  
}
```

Consignas:

1. Aplica la regla de cálculo de eficiencia para estructuras condicionales explicada en clase y determina el costo de la Rama A y de la Rama B en notación Big-O.
2. Calcula la complejidad temporal total del método en el peor caso. ¿De qué parámetro o rama depende este peor caso?

Sección 2: Experimentación y Medición Práctica

EJERCICIO 3

Medición Empírica vs. Medición Teórica (Laboratorio)

Para comprobar el *Principio de Invarianza* y la diferencia entre algoritmos lineales y cuadráticos, implementa en tu entorno de desarrollo dos alternativas para encontrar el valor máximo de un arreglo de enteros:

- **Algoritmo A (Búsqueda Lineal):** Recorre el arreglo una única vez manteniendo una variable con el máximo actual ($O(n)$).
- **Algoritmo B (Comparación Par a Par):** Compara cada elemento contra todos los demás para confirmar si ninguno es mayor ($O(n^2)$).

Consignas:

1. Ejecuta ambos algoritmos con arreglos de números aleatorios para los tamaños $n = 1.000$, $n = 10.000$ y $n = 100.000$.
2. Mide el tiempo de ejecución en milisegundos empleando `System.currentTimeMillis()` o `System.nanoTime()` y completa la siguiente tabla de resultados:

Tamaño de Entrada (n)	Tiempo Algoritmo A — $O(n)$	Tiempo Algoritmo B — $O(n^2)$
n = 1.000		
n = 10.000		
n = 100.000		

1. ¿Qué sucede con el tiempo del Algoritmo B cuando el tamaño de la entrada se multiplica por 10? ¿Cómo se relaciona esto con su notación asintótica?

Sección 3: Notación Asintótica y Órdenes de Crecimiento

EJERCICIO 4

Clasificación y Ordenamiento de Funciones

A continuación se presentan las expresiones teóricas del tiempo de ejecución $T(n)$ obtenidas para cinco algoritmos distintos:

- Algoritmo A: $T_A(n) = 100n^2 + 50n + 12$
- Algoritmo B: $T_B(n) = 2^n + 10$
- Algoritmo C: $T_C(n) = 3n \log_2(n) + 1000$
- Algoritmo D: $T_D(n) = 50000$
- Algoritmo E: $T_E(n) = 2n + 5$

Consignas:

1. Expresa la complejidad asintótica Big-O (O) simplificada de cada uno de los 5 algoritmos (eliminando constantes y términos de menor orden).
2. Ordena los algoritmos de **menor a mayor costo temporal** (del más eficiente al menos eficiente) cuando $n \rightarrow \infty$.

Anexo: Resumen de Reglas de Conteo y Complejidad

Operación / Estructura	Costo Elemental En Peor Caso
Asignación, aritmética, acceso a vector	Constante: $O(1)$
Secuencia de instrucciones	Suma de las operaciones de cada instrucción
Estructura Condicional (<code>if-else</code>)	Condición + $\max(\text{Costo}(\text{Then}), \text{Costo}(\text{Else}))$
Bucle Repetitivo (<code>for</code> , <code>while</code>)	$N \times \text{Costo_Cuerpo}$ (donde N es el número de repeticiones)
Llamada a método / función	$1 + \text{Costo_Ejecucion_Del_Metodo}$